DuesGuard AI

COMPLETE TEAM EXECUTION PLAYBOOK

From zero to integrated Agentic AI MVP in 24 hours

INNOSPRINT 2026 | Team ownership | Build plan | Integration | Testing | Release

THE 24-HOUR OUTCOME
A working web application where an MSME user uploads invoice and supporting PDFs, sees extracted and validated data, views deterministic pending/overdue calculations, observes a real LangGraph multi-agent workflow, reviews evidence gaps, and downloads a case-readiness PDF. This is a hackathon MVP, not a legally certified or fully production-hardened financial product.

How to use this playbook

- Assign one owner per module and folder. Each owner delivers runnable code, tests and a short README.
- Agree the API and data contracts before parallel coding. All members use the same field names and sample data.
- Integrate continuously through pull requests. Do not wait until the final hours to combine the modules.
- The top priority is one complete upload-to-report workflow; optional features come later.

1. Product definition and scope

1.1 Product summary

DuesGuard AI helps Micro, Small and Medium Enterprises organize delayed-payment information. It reads uploaded business documents, extracts invoice and purchase-order fields, validates records, calculates balances using application code, identifies missing or inconsistent evidence, and prepares a reviewable case summary and evidence checklist. Specialized agents coordinate the analysis; the business owner reviews the result before exporting it.

1.2 Problem statement

Invoices, purchase orders, delivery acknowledgements, payment records and buyer communications are often scattered across files. Manually reconciling them takes time and may lead to missed discrepancies, unclear balances or incomplete evidence. Small businesses may not have dedicated finance/legal operations staff to organize this information.

1.3 MVP must-have scope

- Create a case/business profile and upload PDF documents.
- Extract PDF text and return structured fields with nulls/warnings for missing information.
- Validate invoice records, detect duplicates and check purchase-order references.
- Calculate paid, pending and overdue values with deterministic Python logic.
- Run four distinct LangGraph agent nodes against actual case data.
- Generate evidence gaps, chronology and a structured case summary.
- Require human review/approval before final report export.
- Download a PDF case-readiness report.

1.4 Explicit non-goals

- No automatic payment recovery, bank transfer, buyer contact or complaint filing.
- No legal advice, guaranteed eligibility or guaranteed legal outcome.
- No custom foundation-model training, mobile app, microservices or complex cloud setup.
- No claim of production-grade security unless it has been implemented and tested.

Trust rule
Gemini interprets documents and explains findings. Python calculates currency values and date comparisons. Missing or ambiguous data must be marked null/needs_review; never invent it.

2. Team structure and individual ownership

Use four owners. If the team has three members, distribute Member 4's testing and demo duties across the other three.

Member	Owns / responsibilities	Folders and files	Deliverables / definition of done
1 – Project Lead + Agentic AI Lead	Product scope; shared data contract; Gemini integration; LangGraph state, nodes, workflow, event trace and human-review gate.	backend/app/agents/; backend/app/services/llm_service.py; docs/architecture.md; contracts/	Four real agent nodes; typed state; structured findings; workflow callable by backend; tests; agent execution events.
2 – Backend + Data Lead	FastAPI; SQLite; upload/storage; Pydantic schemas; finance calculations; report generation; API documentation.	backend/app/api/; backend/app/db/; backend/app/models/; backend/app/schemas/; backend/app/services/finance.py; report_service.py	Working endpoints; persisted documents/analysis; deterministic calculations; Swagger; backend tests; no committed secrets.
3 – Frontend Lead	React layout; navigation; upload UX; dashboard; invoice table; agent activity; evidence review; report approval/download.	frontend/src/pages/; components/; api/; styles/	Complete API-connected user flow; loading/error/empty states; all buttons functional; npm build passes.
4 – QA + Sample Data + Release Lead	Synthetic documents; expected-value manifest; integration tests; bug triage; demo script; README; release and backup plan.	sample_data/; tests/e2e/; docs/demo-script.md; docs/release-checklist.md	Repeatable synthetic case; test evidence; defect list with owners; rehearsed demo; fresh-clone run instructions.

2.1 Member 1 – exact task list

1. Agree the shared JSON schemas with Members 2 and 3 before implementing the agents.
2. Build a Gemini wrapper. Load GEMINI_API_KEY only from backend environment variables.
3. Extract document_type, invoice_number, buyer, seller, invoice_date, due_date, amount, tax, PO reference, payment terms, source page and warnings.
4. Define shared graph state: case_id, documents, extracted_records, validation_findings, evidence_findings, finance_summary, timeline, errors and approval_status.
5. Implement nodes: extract_documents, validate_invoices, verify_evidence, calculate_finances (calls deterministic finance service), prepare_case_report.
6. Connect nodes in LangGraph and make failures visible. Add a human-review/approval gate before report finalization.
7. Expose ordered agent events and structured outputs to the API.
8. Write tests using Member 4's synthetic case and document the graph invocation function.

Done when: the graph runs on uploaded document data; each node returns structured output; financial totals come from the Python finance service; errors are visible; report approval is required.

2.2 Member 2 – exact task list

9. Create FastAPI app, settings, CORS and /api/health.
10. Create SQLite models for cases, documents, invoices, payments, analysis_runs and findings.

11. Implement PDF upload with file type/size checks and safe generated filenames.
12. Implement API routes exactly as listed in Section 4.
13. Implement finance.py using Decimal for currency arithmetic; avoid float for money.
14. Build dashboard statistics from persisted data, not constants.
15. Build ReportLab report generation from approved analysis data.
16. Write endpoint and calculation tests.

Done when: backend starts from a clean clone; Swagger shows agreed endpoints; uploads persist; calculations are reproducible; database survives restart.

2.3 Member 3 – exact task list

17. Create React + Vite app and use VITE_API_BASE_URL for the backend URL.
18. Build shared layout, sidebar/navigation and responsive styling.
19. Create Dashboard page using GET /api/dashboard.
20. Create Documents page with PDF upload, progress and document list.
21. Create Analysis page with agent events, findings, warnings and evidence gaps.
22. Create Invoices page with invoice, due date, amount, paid, outstanding and status columns.
23. Create Case Report page with review/approval and PDF download.
24. Use real API responses; show loading, empty, success and error states.

Done when: a judge can complete the full scenario without Swagger or developer intervention; npm run build passes; no API key or metrics are hardcoded.

2.4 Member 4 – exact task list

25. Create one synthetic business case: three invoices, one purchase order, delivery proof, one payment record and one deliberately missing or contradictory item.
26. Create a manifest of expected extracted values and expected calculations.
27. Test invalid PDF, missing fields, duplicate invoice, partial payment, date boundaries and Gemini failure.
28. Log each bug with severity, owner, steps, expected/actual result and screenshot/log.
29. Run end-to-end tests after every integration checkpoint.
30. Prepare demo script, speaker order, architecture explanation and backup screenshots/video.
31. Write README run instructions and confirm no secrets or real customer data are committed.

Done when: a repeatable synthetic case exists; critical tests pass; known issues are documented; a fresh clone can be run from README.

3. Shared folder structure

```
DuesGuardAI/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── core/config.py
│   │   ├── db/session.py
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── api/routes/{cases,documents,analysis,dashboard,reports}.py
│   │   ├── services/{pdf_extractor,llm_service,finance,report_service}.py
│   │   └── agents/{state,nodes,graph}.py
│   ├── storage/uploads/       # ignored by Git
│   ├── tests/
│   ├── .env.example
│   └── requirements.txt
├── frontend/src/{api,components,pages,hooks}/
├── sample_data/
├── tests/e2e/
├── docs/{architecture,api-contract,demo-script,release-checklist}.md
├── .gitignore
└── README.md
```

Folder ownership: Member 1 owns agents and LLM orchestration; Member 2 owns API/database/finance/report services; Member 3 owns frontend/src; Member 4 owns sample_data, end-to-end tests and release docs.

4. Integration contract – freeze before parallel coding

All members must use these names and response shapes. If a field changes, update docs/api-contract.md and tell the team before coding against it.

4.1 Core entities

Entity	Required MVP fields	Notes
Case	case_id, business_name, created_at, status	One delayed-payment matter.
Document	document_id, case_id, filename, document_type, uploaded_at, extraction_status	Do not expose local server paths to browser.
Invoice	invoice_id, case_id, document_id, invoice_number, buyer, seller, invoice_date, due_date, total_amount, currency, po_reference	Money as Decimal/string; missing fields nullable.
Payment	payment_id, invoice_id, payment_date, amount, reference	Partial payments allowed.
Finding	finding_id, case_id, agent_name, severity, code, message, source_document_ids, status	Every finding points to source docs or is marked needs_review.

4.2 API routes and owners

Method + route	Owner	Request / response behavior
GET /api/health	Member 2	{"status":"ok","version":"0.1.0"}
POST /api/cases	Member 2	{"business_name":"XYZ Enterprises"} → {"case_id":"...","status":"created"}
POST /api/cases/{case_id}/documents	Member 2	multipart form: file, document_type → document_id, filename, status
GET /api/cases/{case_id}/documents	Member 2	Returns uploaded document list and extraction status.
POST /api/cases/{case_id}/analysis	Members 1 + 2	Runs graph; returns analysis_id, summary, findings, agent_events and approval_status.
GET /api/cases/{case_id}/analysis/{analysis_id}	Members 1 + 2	Returns saved analysis and events.
GET /api/dashboard?case_id=...	Member 2	Returns total_pending, overdue, paid, invoice_count.
POST /api/cases/{case_id}/approve	Members 2 + 3	Accepts analysis_id and reviewer_notes; updates approval status.
POST /api/cases/{case_id}/reports	Member 2	Accepts analysis_id; returns PDF file/download response.

4.3 Example analysis response


```
{
 "analysis_id":"an_001", "case_id":"case_001", "status":"completed",
 "summary":{
   "total_invoiced":"100000.00", "total_paid":"0.00",
   "total_pending":"100000.00", "total_overdue":"80000.00", "currency":"INR"
 },
 "findings":[{
   "agent_name":"evidence_verification", "severity":"warning",
   "code":"MISSING_DELIVERY_PROOF",
   "message":"Delivery acknowledgement was not found for INV-102.",
   "source_document_ids":["doc_inv_102"], "status":"needs_review"
 }],
 "agent_events":[
   {"step":"document_extraction","status":"completed","message":"3 invoices extracted"},
   {"step":"invoice_validation","status":"completed","message":"No duplicate invoice numbers"},
   {"step":"evidence_verification","status":"completed","message":"1 evidence gap found"},
   {"step":"case_preparation","status":"completed","message":"Draft report prepared"}
 ],
```

```
 "approval_status":"pending"
}
```

4.4 Contract rules

- Money travels as decimal strings such as '50000.00', not floating-point values.
- Dates use YYYY-MM-DD; timestamps use ISO 8601 with timezone.
- Missing or uncertain fields are null plus warnings.
- Errors use a consistent JSON shape: {"detail":"message","code":"ERROR_CODE"}.
- Frontend accesses data only through APIs, never SQLite or local upload paths.
- Analysis is persisted so refresh/reopen does not erase it.
- Report must not be labelled approved until the user confirms review.

4.5 Agent/backend interface

```
# Member 1 exposes this callable service:
def run_case_analysis(case_id: str, documents: list[dict]) -> dict:
    # Run LangGraph; return JSON-serializable structured result.
    ...

# Member 2 invokes and persists it:
result = run_case_analysis(case_id, normalized_documents)
save_analysis_result(db, case_id, result)
return result
```

5. GitHub collaboration and merge process

Use one shared repository and one branch per member. GitHub Flow recommends descriptive branches, commits, pull requests, review and merge; keep main runnable.

```
# Lead initializes repo
git init
git add .
git commit -m "chore: initialize DuesGuard AI"
git branch -M main
git remote add origin <REPOSITORY_URL>
git push -u origin main

# Each member clones once and creates own branch
git clone <REPOSITORY_URL>
cd DuesGuardAI
git checkout -b feat/ai-agents       # Member 1
git checkout -b feat/backend-api     # Member 2
git checkout -b feat/frontend-ui     # Member 3
git checkout -b test/demo-readiness  # Member 4

# Commit and push small changes
git add .
git commit -m "feat: add document upload endpoint"
git push -u origin <branch-name>
```

- Do not have two people edit the same file without coordination.
- Open a PR for each module; include changes, run instructions and test evidence.
- Lead/integrator reviews and merges. Do not push untested code directly to main.
- Merge small working increments; resolve conflicts before final integration.
- Ignore .env, .venv, node_modules, uploads, database files and generated secrets.

```
.gitignore:
.env
.env.*
!.env.example
.venv/
__pycache__/
node_modules/
dist/
backend/storage/uploads/
```

```
*.db
*.sqlite
```

6. 24-hour management timeline and checkpoints

Time	Parallel team work	Exit gate
00:00–01:00	Kickoff, repo, owners, scope, schemas and API contract.	Everyone clones; branches and contract agreed.
01:00–03:00	AI skeleton; API skeleton; frontend shell; synthetic data.	API health works; React starts; graph imports; demo docs ready.
03:00–06:00	PDF upload/extraction; Gemini structured extraction; DB; upload UI.	One PDF yields validated structured fields visible through API/UI.
06:00–08:00	Finance calculations, invoice validation, sample tests.	Pending/overdue totals pass deterministic tests.
08:00–11:00	LangGraph nodes and analysis API; agent trace UI.	Graph runs on actual data and returns structured findings.
11:00–13:00	Evidence gap logic, chronology and report generation.	Missing evidence appears; draft PDF is produced.
13:00–16:00	Frontend pages and API integration.	User can upload, analyze and view results without Swagger.
16:00–18:00	End-to-end integration, persistence and review/approval.	Upload → analysis → review works on shared main.
18:00–20:00	QA, failure tests, bug fixes and UI polish.	No P0 bug; known limitations recorded.
20:00–22:00	Release candidate, README and demo rehearsal.	Fresh-clone smoke test passes; no secrets in repo.
22:00–24:00	Final demo, pitch, backup capture and submission.	Stable demo and speaker order confirmed.

Management rule
At each checkpoint ask: what works now, what is blocked, who owns the fix, and what is the next smallest deliverable? If a checkpoint is missed, remove optional scope. Do not steal time from integration and rehearsal.

7. Exact sequence to combine all four members' work

32. Create repository, folders and .gitignore before parallel coding. Push the skeleton to main.
33. Publish API contract and JSON examples. Members 1, 2 and 3 agree field names before implementing.
34. Member 4 publishes the synthetic documents and expected-results manifest. Everyone uses the same dataset.
35. Member 2 builds /api/health, case creation and document upload. Member 3 connects the upload screen to these endpoints.
36. Member 1 implements the LangGraph workflow as a standalone callable function accepting case_id and normalized document data. It must not depend on FastAPI request objects.
37. Member 2 exposes POST analysis endpoint, calls the graph service and persists its output.
38. Member 3 connects Analyze button and renders actual summary, agent_events and findings from the endpoint.
39. Member 2 creates PDF report service from approved analysis data. Member 3 connects the download action.
40. Member 4 runs the complete demo flow on shared main and files defects with severity, owner and reproduction steps.
41. Integrator merges only tested PRs. After each merge run backend tests, frontend build and one smoke test.
42. Freeze features at Hour 20. Only fix bugs, rehearse and prepare backups afterwards.

7.1 Integration handoff checklist

From → To	What must be handed over	How receiver verifies it
Member 1 → Member 2	Graph callable function, typed input/output, example result, errors and tests.	Call from analysis service with sample docs; assert JSON keys.
Member 2 → Member 3	Swagger URL, API routes, sample request/response, CORS and base	Frontend API client can upload,

	URL.	analyze and download.
Member 4 → All	Synthetic files, manifest, expected extracted fields and calculations.	Everyone runs same test case and compares values.
All → Integrator	Branch, PR, run instructions, test output and known limitations.	Fresh checkout/build and smoke test before merge.

8. Local setup and run commands (Windows)

8.1 Backend

```
cd DuesGuardAI
cd backend
py -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Copy-Item .env.example .env
# Add GEMINI_API_KEY locally in backend/.env
uvicorn app.main:app --reload --port 8000
```

Backend: http://localhost:8000 | Swagger: http://localhost:8000/docs

8.2 Frontend – second terminal

```
cd DuesGuardAI\frontend
npm install
# frontend/.env.local: VITE_API_BASE_URL=http://localhost:8000
npm run dev
```

Vite commonly serves at http://localhost:5173; use the exact URL shown by the terminal.

8.3 Environment examples

```
# backend/.env.example
GEMINI_API_KEY=replace_me
DATABASE_URL=sqlite:///./duesguard.db
UPLOAD_DIR=./storage/uploads
MAX_UPLOAD_MB=15
ALLOWED_ORIGINS=http://localhost:5173

# frontend/.env.example
VITE_API_BASE_URL=http://localhost:8000
```

Never use VITE_GEMINI_API_KEY. Vite-exposed variables are bundled into browser code. Keep the Gemini key on the backend.

9. Testing and acceptance criteria

Test	Expected result	Priority
Upload valid text PDF	File stored; document row created; extraction can run.	P0
Unsupported/oversized file	Clear validation error; no false success.	P0
Missing due date	due_date=null; warning/needs_review; no invented date.	P0
Duplicate invoice number	Finding references both documents.	P0
Partial payment	Outstanding = invoice amount − recorded payments.	P0
Multiple invoices	Dashboard totals equal sum of invoice balances.	P0
Missing delivery proof	Evidence agent flags the gap and affected invoice.	P0
Gemini unavailable	Recoverable error; no fabricated analysis.	P0
Report before approval	Blocked or marked draft, never falsely approved.	P0
Refresh analysis page	Saved analysis is retrieved from database.	P1
Fresh clone run	README commands start both applications.	P1

9.1 Synthetic demo truth table

Invoice	Amount	Paid	Outstanding	Expected status
INV-101	50000.00	0.00	50000.00	Overdue
INV-102	30000.00	0.00	30000.00	Overdue
INV-103	20000.00	0.00	20000.00	Not yet due
TOTAL	100000.00	0.00	100000.00	80000.00 overdue

Set the synthetic invoice dates relative to the actual demo date so the statuses are true. Do not hardcode the overdue total; calculate it from due dates and the selected as-of date.

10. Production-readiness: honest scope

Important distinction
A 24-hour hackathon can deliver a production-style end-to-end MVP, but not a responsibly production-hardened financial/legal document product. Do not claim the system is production ready solely because the demo works.

Area	Hackathon MVP	Before real production use
Authentication	Demo session/basic login if time permits.	Verified identity, secure sessions, MFA option, password reset, rate limiting.
Authorization	Single demo case; avoid cross-user access.	Tenant isolation, role-based access, object-level authorization tests.
File security	PDF type/size checks, safe names, private local storage.	Malware scanning, encrypted object storage, retention/deletion policy.
Secrets	Environment variables; .env excluded from Git.	Managed secret store, rotation, least privilege.
Database	SQLite local demo.	Managed PostgreSQL, migrations, backups and restore tests.
AI quality	Schema validation, nulls for missing fields, visible warnings.	Evaluation set, prompt/version tracking, hallucination tests, confidence thresholds, human review.
Financial accuracy	Decimal arithmetic and unit tests.	Audit trail, reconciliation controls, independent validation and finance-domain review.
Agent state	In-process graph; optional local checkpointer.	Persistent checkpointer and durable job execution for resumable workflows.
Observability	Structured logs and visible agent events.	Centralized logs, metrics, tracing, alerting and error monitoring.
Deployment	Local or single demo hosting environment.	HTTPS, staging/production separation, CI/CD, rollback, monitoring.
Privacy/legal	Synthetic demo data; no legal advice claim.	Privacy and legal review, consent, retention policy, jurisdiction-specific review.

10.1 Release stages

43. Development: each owner runs module tests locally.
44. Integration: PRs merge after review and checks.
45. Release candidate: feature freeze; run acceptance tests on one clean dataset.
46. Demo deployment: deploy only a smoke-tested version and keep local fallback.
47. Post-hackathon hardening: complete the security, privacy, reliability, audit and legal-review work before real customer use.

11. Risk register and fallback decisions

Risk	Early warning	Mitigation
Gemini quota/network failure	Timeout or quota error.	Retry once; preserve extracted text; show pending/manual review; use labelled synthetic fixture only as demo fallback.

Scanned PDF	PDF text extraction returns empty.	Prioritize text PDFs; detect and explain OCR is outside MVP scope.
LangGraph delay	Graph not callable by Hour 10.	Keep four real nodes but simplify; expose one synchronous run_case_analysis function.
API contract mismatch	Frontend sees missing/renamed fields.	Freeze contract; share sample JSON; run smoke test after each merge.
Merge conflicts	Multiple people change same files.	Folder ownership, small PRs, integrator review.
PDF report delay	Report not ready by Hour 18.	Generate clean text-first PDF with summary, timeline, findings and source names.
Time overrun	Critical milestone late by >30 minutes.	Cut optional auth, OCR, animations, charts and deployment complexity.

12. Demo plan and speaker assignments

Time	Speaker/action	What judges should see
0:00–0:25	Member 4/lead: explain delayed-payment problem.	Clear user pain and target MSME.
0:25–0:50	Member 3: upload synthetic documents.	Working UI and actual file upload.
0:50–1:25	Member 1: run analysis and show agent events.	Four specialized agents processing actual case data.
1:25–1:55	Member 2: show calculations and validation.	₹1,00,000 pending; ₹80,000 overdue based on sample dates.
1:55–2:25	Member 1/3: evidence gaps and review.	Traceable missing evidence and human oversight.
2:25–3:00	Member 4: export PDF and close.	Useful case-readiness report and product value.

- Member 1 explains LangGraph and agent coordination.
- Member 2 explains deterministic finance calculations, API and persistence.
- Member 3 demonstrates the user workflow and dashboard.
- Member 4 explains test coverage, synthetic demo, limitations and next steps.

13. Final release checklist

- All member PRs merged into main after review.
- README has prerequisites, setup, environment variables and run commands.
- Backend health endpoint works and frontend uses correct API base URL.
- Synthetic documents are clearly labelled; no real customer documents used in public demo.
- One complete upload → extraction → agents → calculation → evidence → approval → PDF flow passes.
- Financial results match the test manifest.
- No fake agent output, fake metrics or nonfunctional buttons.
- No API keys, .env, private documents or database secrets in Git.
- Known limitations are stated honestly.
- Demo rehearsed twice; backup screenshots/video ready.

14. Official technical references

Use official documentation for implementation details and check current package/version guidance at setup time.

- **Vite – Getting Started:** Scaffolding, development server and production build. https://vite.dev/guide/
- **Gemini API – Structured Outputs:** Schema-constrained JSON output and validation. https://ai.google.dev/gemini-api/docs/structured-output
- **LangGraph – StateGraph reference:** Shared graph state, node behavior and compilation. https://reference.langchain.com/python/langgraph/graph/state

- **LangGraph – Persistence:** Checkpointing and resumable workflows. https://docs.langchain.com/oss/python/langgraph/persistence
- **FastAPI – SQL databases:** Database integration patterns and SQLite examples. https://fastapi.tiangolo.com/tutorial/sql-databases/
- **GitHub – GitHub Flow:** Branches, commits, pull requests and merging. https://docs.github.com/en/get-started/using-github/github-flow

15. Project lead – first 30 minutes

48. Create GitHub repository and invite all members.
49. Create folder structure and .gitignore; push initial commit.
50. Assign folder ownership and add this playbook to docs/.
51. Freeze core entities, API routes and example response in Section 4.
52. Ask Member 4 to create the synthetic test pack immediately.
53. Create a shared issue board: To do, In progress, Blocked, Review, Done.
54. Set checkpoints at Hours 3, 6, 8, 11, 16, 18, 20 and 22.
55. Require blockers to be raised immediately, not at the next meeting.

Final operating principle

Build the smallest complete product first. Parallelize independent modules, integrate against fixed contracts, test continuously, and freeze features early. A reliable end-to-end case is more convincing than many unfinished features.